Practice Midterm

Question 1

6 challenges in microservice architecture

- Distributed complexity: many moving parts, network calls, partial failures.
- Latency & reliability: remote calls add delay; need timeouts, retries, circuit breakers.
- Data consistency: each service owns its DB → sagas/eventual consistency, hard transactions.
- Observability: need centralized logging, tracing, metrics to debug cross-service flows.
- Testing & environments: integration testing, contract testing, test data setup is harder.
- Operational overhead: CI/CD per service, versioning, configuration, security, service discovery.

Question 2 – Modular monolith vs Microservices

Legend: 1 = component based monolith fits better, 2 = microservices fits better, 3 = both.

Item	Requirement	Answer (1/2/3)	Why
a	Easy to add new functionality	1	Lower coordination overhead inside one codebase.
b	Architecture should be simple	1	Fewer distributed concerns.
c	Easy to try new technology	2	One service can adopt new tech independently.
d	Performance very fast	1	In-process calls, no network hops.
e	Easy to reuse existing functionality	1	Shared components/modules without remote calls.
f	Deployment should be very fast	1	Single deploy pipeline; fewer deploy units to coordinate.
g	Scale different functionality differently	2	Scale services independently.
h	Easy to use different types of databases	2	Polyglot persistence per service.
i	Scope of functionality is large	2	Split by bounded contexts/teams.
j	Multiple agile/scrum teams needed	2	Clear team/service ownership.

Question 3 – Correct statements

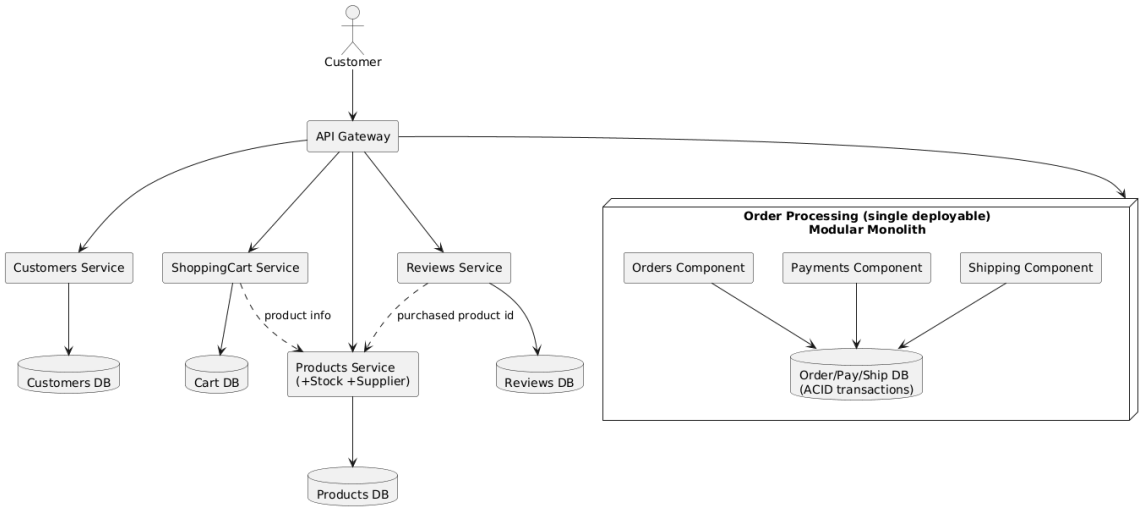
Correct statements to circle: 1, 3, 5, 8, 10, 13.

- 1 ✓ Custom router can return a String or list/collection of channel names (Strings).
- 3 ✓ Spring Data repositories are typically interfaces (Spring creates implementation).
- 5 ✓ In MongoDB we usually avoid joins/lookups in hot paths; prefer embedding/denormalization.
- 8 ✓ Value objects are immutable in DDD.

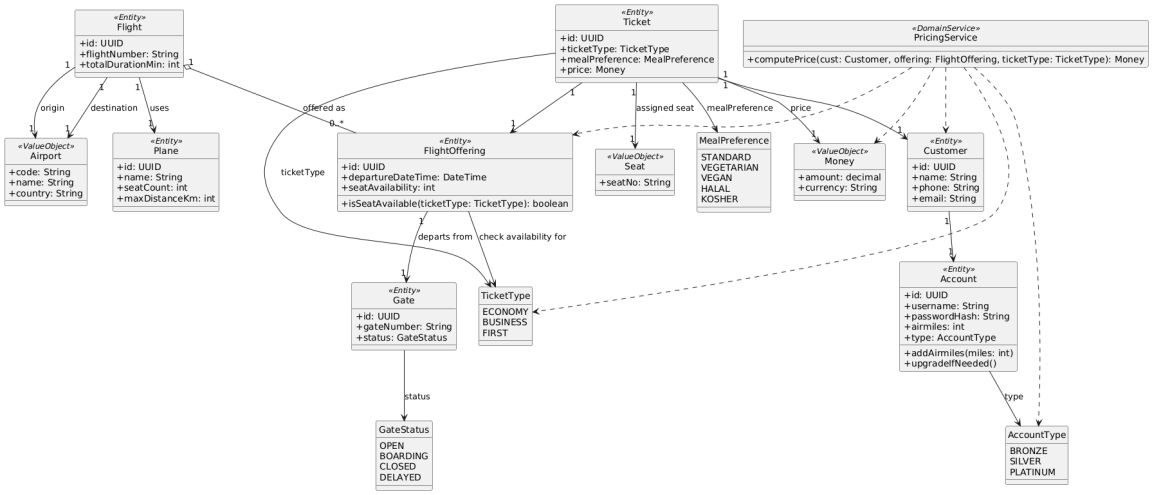
- 10 ✓ Partitioning (sharding) and replication can be combined.
- 13 ✓ Cassandra data model is query-driven: design tables based on required queries.

Question 4 – Webshop architecture design

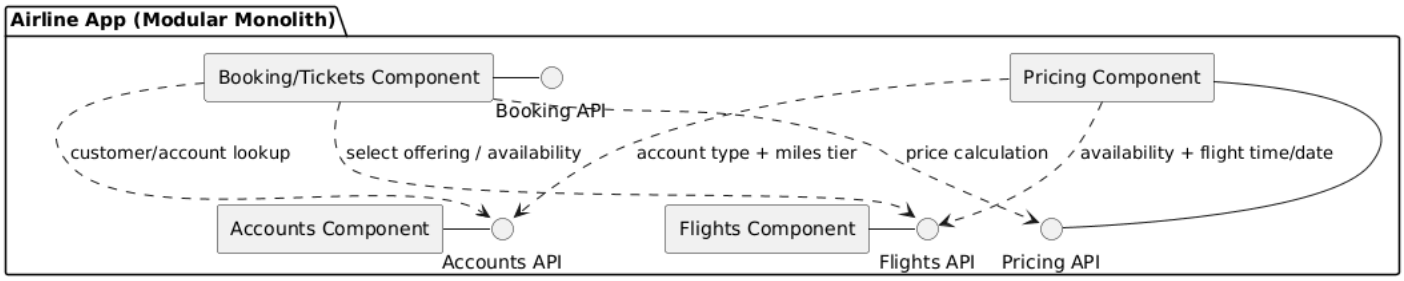
- Use microservices for: Customers, Products (+Stock+Supplier), ShoppingCart, Reviews so each team can own and scale independently.
- Group Orders + Payments + Shipping into ONE deployable (modular monolith) with a single ACID database to guarantee strict consistency and high performance.
- Expose everything via an API Gateway (routing, auth, rate limit).
- Within the Orders/Payments/Shipping deployable: components talk via in-process calls + DB transactions (fast + consistent).
- Across microservices: use synchronous REST for simple queries, and async events (Kafka/RabbitMQ) for eventual-consistency updates (e.g., product viewed, review created).
- Each microservice has its own database to avoid tight coupling; product-related data stays in the Products service boundary.



Question 5a – Airline domain class diagram



Question 5b – Airline modular monolith components



Non-domain classes per component (examples):

- Accounts: AccountController, AccountService, AccountRepository
- Flights: FlightController, FlightService, FlightRepository
- Booking/Tickets: TicketController, BookingService, TicketRepository
- Pricing: PricingService (domain), PricingFacade/Controller (non-domain)

Question 6 – Cassandra data model (query-driven)

- Goal 1 page: grades + course info for ONE student → partition by studentId.
- Goal 2 page: grades + student info for ONE course → partition by courseNumber.
- Use 2 denormalized tables (fast single-partition reads; no joins).

Table A: grades_by_student (Partition key: studentId)

```
CREATE TABLE grades_by_student (
  studentId int,
  courseNumber text,
  courseName text,
  grade text,
  PRIMARY KEY ((studentId), courseNumber)
);
```

studentId (PK)	courseNumber (CK)	courseName	grade
5566	CS590	Software Architecture	A+
5566	CS525	Advanced Software Development	B
5532	CS590	Software Architecture	A-

Table B: grades_by_course (Partition key: courseNumber)

```
CREATE TABLE grades_by_course (
  courseNumber text,
  studentId int,
  firstName text,
  lastName text,
  grade text,
  PRIMARY KEY ((courseNumber), studentId)
);
```

courseNumber (PK)	studentId (CK)	firstName	lastName	grade
CS590	5566	Frank	Brown	A+
CS590	5532	John	Doe	A-
CS525	5566	Frank	Brown	B

Question 7 – ESB related to SCI principles

An Enterprise Service Bus (ESB) expresses the SCI idea of **integration and wholeness:** many independent services become one coordinated system when they connect through a shared integration layer (routing, transformation, mediation). Instead of each service building ad-hoc point-to-point links, the ESB provides a single coherent "nervous system", so information flows in an orderly way and the whole enterprise behaves more like one organism than many fragments.

It also matches the SCI principle often explained as **"water the root to enjoy the fruit."** If we strengthen the root (a well-designed integration backbone: standardized messages, monitoring, security, and governance), then every connected application benefits automatically: easier change, more reliability, and faster delivery. By investing in the root layer, we reduce friction across the entire organization—supporting growth of all "branches" (business capabilities) without having to fix integration separately for each new feature.